

Two Steps Are All You Need: Efficient 3D Point Cloud Anomaly Detection with Consistency Models

Pranav A¹ Shashank B¹ Pranav Siddappa¹ Dominik Seuss²
Minal Moharir¹ Subramanya KN¹

¹R.V. College of Engineering ²Technical University of Applied Sciences Würzburg-Schweinfurt

¹{pranava.cs21, shashankb.cs21, pranavs.cs21, minalmoharir, subramanyakn}@rvce.edu.in

²dominik.seuss@thws.de

Abstract

*Diffusion models are rapidly redefining 3D anomaly detection in point cloud data. As 3D sensing becomes integral to modern manufacturing, reliable anomaly detection is essential for high-throughput quality assurance and process control. Yet practical deployment on resource-constrained, latency-critical systems remains limited. Existing methods are often computationally prohibitive or unreliable in complex, unmasked regions, and diffusion pipelines are inherently bottlenecked by iterative denoising. In this work, we address this bottleneck by reformulating reconstruction-based anomaly detection through consistency learning, enabling direct prediction of anomaly-free geometry in one or two network evaluations. We further introduce a novel hybrid loss formulation that explicitly enforces reconstruction toward clean data. This design substantially reduces inference cost, achieving up to $80\times$ faster runtime than the current state-of-the-art method, without GPU acceleration, while preserving strong detection performance. It outperforms R3D-AD on Anomaly-ShapeNet with **76.20%** I-AUROC and remains competitive on Real3D-AD with **72.80%** I-AUROC, enabling efficient, low-latency anomaly detection on resource-constrained platforms, including drones, smart industrial cameras, and other edge devices.*

1. Introduction

3D point cloud anomaly detection is a fundamental aspect of data analysis having far reaching applications, particularly in quality assurance and control. Existing 3D methods—particularly diffusion-based methods—often assume the availability of GPU-class hardware and offline processing, thus confining them to academic research. To the best of our knowledge, at the time of writing, no approach has been explicitly designed for efficient, low-

latency operation on resource-constrained edge systems.

Among emerging approaches in the 3D point cloud domain, we focus specifically on the diffusion-based paradigm, where reconstruction-oriented generative methods have shown considerable promise. While several innovative approaches have been proposed for 3D point cloud generation using diffusion models [5, 12], their application to anomaly detection remains nascent. At the time of writing, only a single work [22] has explored this direction, and even this work fails to provide an industrially adaptable solution.

In this work, we propose CM3D-AD (Consistency Models based 3D Anomaly Detection), which explicitly solves the latency bottleneck seen in prior approaches, and thus helps bridge the gap between academic advances and industrial deployment. CM3D-AD leverages conditionally guided consistency models (CMs) [16] to reconstruct anomaly-free point clouds in real time, without requiring hardware acceleration.

To summarize, our main contributions are as follows:

- i. We identify the efficiency bottleneck in diffusion-based methods, and reformulate anomaly detection as a single-step manifold projection problem, to be solved via consistency learning.
- ii. We introduce CM3D-AD, leveraging conditionally guided consistency models to directly predict anomaly-free geometry in one or two network evaluations.
- iii. We demonstrate competitive detection performance against current state-of-the-art models on both Anomaly-ShapeNet [10] and Real3D-AD [11], while achieving up to $80\times$ faster inference without hardware acceleration, enabling efficient edge deployment.
- iv. We further benchmark the proposed model alongside R3D-AD on two representative edge platforms—Raspberry Pi 4 and Jetson Nano (2 GB)—and show that our model outperforms R3D-AD on all efficiency

metrics, making it substantially more suitable for deployment on resource-constrained edge systems.

2. Related Works

2.1. 2D Anomaly Detection

Anomaly detection in 2D images has been widely studied, particularly in industrial inspection and medical imaging. Existing methods are commonly grouped into flow-based, memory-based, and reconstruction-based families.

Flow-based models estimate feature likelihoods to identify anomalies. Representative methods include CFLOW-AD [7], which uses conditional flows for resource-efficient detection; U-Flow [17], which improves segmentation via a U-shaped transformer design; and FastFlow [19], which improves throughput by applying flows directly to deep features. Memory-based approaches, such as FAPM [9] and PatchCore [14], compare test features against a memory bank of normal patterns.

Reconstruction-based methods detect anomalies through reconstruction residuals. Notable advances include perceptual/SSIM-driven objectives [2], adversarial reconstruction as in DRÆM [20], and inpainting-based strategies such as RIAD [21]. Despite strong results, these paradigms can remain sensitive to data variation, feature transferability, and subtle defect morphology.

2.2. 3D Anomaly Detection

In 3D point cloud anomaly detection, prior methods such as M3DM [18], CPMF [3], IMRNet [10], and PatchCore [14] largely rely on memory-bank matching or iterative restoration for anomaly localization.

Recent extensions of diffusion modeling to 3D data have driven substantial progress in point cloud analysis. Luo *et al.* [12] introduced a probabilistic point cloud generation framework relevant to downstream anomaly detection, while PNI [1] improved industrial 3D detection by combining spatial coordinates with local neighborhood features. A large-scale 3D anomaly detection benchmark and associated self-supervised framework (IMRNet) for synthetic and real defects were presented in [10]. Furthermore, R3D-AD [22] advances diffusion-based 3D reconstruction by operating directly on point clouds with a PointNet backbone [13], avoiding voxelization and preserving permutation invariance. It iteratively predicts point-wise corrections and introduces Patch-Gen for defect simulation in training data, improving accuracy over earlier 3D approaches.

2.3. Consistency Models

Consistency models build on the probability flow ordinary differential equation (PF-ODE) [15], whose solution trajectories at any timestep t are distributed according to Eq. (1). This ODE defines a trajectory that transitions smoothly

from a data distribution to a noise distribution.

$$d\mathbf{x}_t = \left[\mu(\mathbf{x}_t, t) - \frac{1}{2} \sigma(t)^2 \nabla \log p_t(\mathbf{x}_t) \right] dt, \quad (1)$$

Furthermore, Song *et al.* [16] provides an empirical PF ODE by taking $\mu(\mathbf{x}_t, t) = 0$ and $\sigma(t) = \sqrt{2t}$, given by:

$$\frac{d\mathbf{x}_t}{dt} = -t s_\phi(\mathbf{x}_t, t), \quad (2)$$

where $s_\phi(\mathbf{x}_t, t)$ is a score model trained to estimate the score function $\nabla \log p_t(\mathbf{x}_t)$. Consistency models are designed for efficient inference by learning a direct mapping from any point on the noise trajectory to clean data. This bypasses iterative denoising, enabling single-step or few-step sampling and significantly reducing inference time.

In 3D point cloud anomaly detection, this provides a key advantage: fast, reliable reconstructions without significant compute, making the approach highly suitable for latency-critical applications on computationally constrained platforms.

3. Methodology

3.1. Overview

We formulate 3D point cloud anomaly detection as a single-step manifold projection problem and leverage conditionally guided consistency models to reconstruct anomaly-free point clouds in one or two network evaluations, enabling real-time on-device inference. We augment the dataset with Patch-Gen [22], which introduces localized perturbations into point clouds. During training, the augmented point cloud is fed to the consistency model, which learns to map noisy inputs at any timestep t to anomaly-free point clouds. During testing, the anomalous point cloud is passed through the trained consistency model to obtain a clean reconstruction. We then compute local reconstruction errors between input and reconstruction, and use them as anomaly scores to detect and localize anomalous regions.

3.2. Anomaly Simulation Strategy

To address the absence of anomalous samples in the training set, we adopt Patch-Gen [22] to synthesize anomalous point clouds from normal instances. Patch-Gen injects localized geometric perturbations into the training point clouds according to the following equation:

$$\mathcal{P}_n = \mathcal{P}_n + S \cdot \text{normalize}(\mathcal{P}_n - \mathcal{P}_v) \odot \mathcal{T}, \quad (3)$$

where $\mathcal{P}_v$ is a randomly sampled pivot point, S is a predefined scaling hyper-parameter, and $\mathcal{T}$ is a translation matrix originating from a Gaussian distribution.

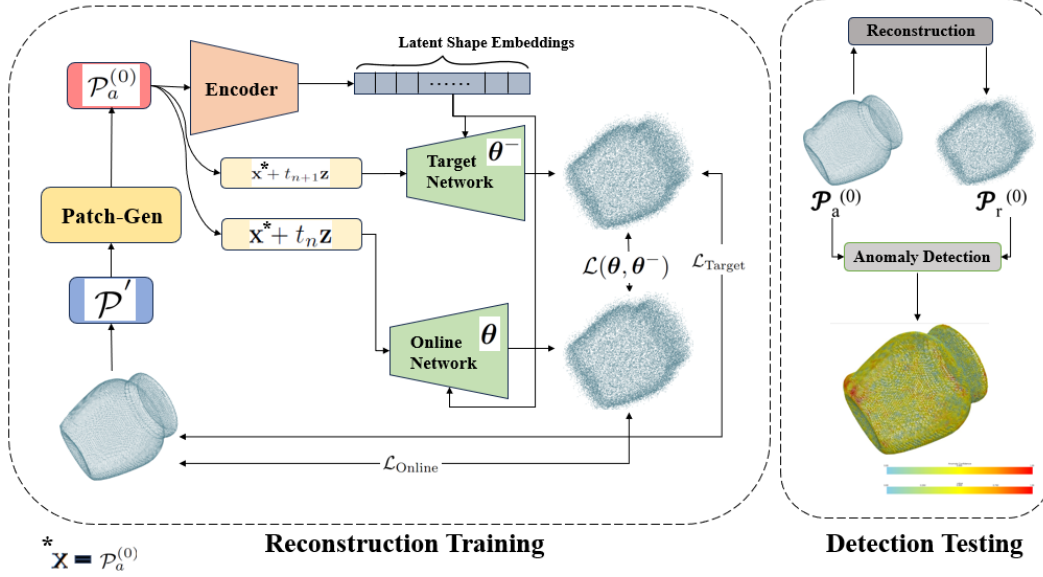


Figure 1. Overview of the proposed consistency-based anomaly detection framework. **Training Phase:** (left), the model learns to denoise perturbed point clouds using the hybrid loss to enforce anomaly-free reconstruction and cross-timestep consistency. **Testing Phase:** The denoising network output is compared with the input for anomaly detection. The resulting heatmap highlights the localized bulge, with anomaly-score confidence color-coded from sky-blue/green (normal) to red (anomalous).

3.3. Latent Shape Encoder

The anomalous point cloud is passed through a jointly trained feature encoder such as a PointNet encoder [13] to obtain latent shape embeddings. These shape embeddings are passed to the model as context, thus providing it with information regarding the overall geometry and shape of the point cloud during the reconstruction phase. Furthermore, due to the constraints of the training paradigm as well as the bottleneck of the latent dimension, these shape embeddings learn to encode global geometry while remaining insensitive to local perturbations i.e. anomalies.

3.4. Features of Consistency Models

3.4.1. Noise Schedule

We follow Karras *et al.* [8] to determine the noise schedule using a non-linear interpolation in noise space. The discrete time indices $t_i \in [\epsilon, T]$ are defined using a curvature parameter $\rho > 0$ as:

$$t_i = \left(T^{1/\rho} + \frac{i}{N-1} \cdot (\epsilon^{1/\rho} - T^{1/\rho}) \right)^\rho, \quad (4)$$

$$i = 0, 1, \dots, N-1$$

In our experiments, we use $\rho = 7$, following current literature [16], which concentrates more steps at lower noise levels. Given a clean point cloud x_0 , we sample a noisy point cloud x_i , using the Karras schedule, as shown below:

$$x_i = x_0 + t_i \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (5)$$

3.4.2. Consistency Training

Our approach follows the consistency training paradigm, where the model is trained in isolation without teacher-student distillation from a pre-trained diffusion model. As shown in Figure 1, training uses two networks: an online network f_θ and a target network f_{θ^-} . The online network f_θ is implemented as a PointwiseNet with six ConcatSquash-Linear layers. The target network shares the same architecture and is updated via an exponential moving average (EMA), governed by:

$$\theta^- \leftarrow \text{stopgrad}(\mu(k)) \cdot \theta^- + (1 - \mu(k)) \cdot \theta \quad (6)$$

Keeping in mind the architectural boundary condition described in [16], the consistency model is parameterized as follows:

$$f_\theta(x, t, c) = c_{\text{skip}}(t) \cdot x + c_{\text{out}}(t) \cdot F_\theta(c_{\text{in}} \cdot x, t, c) \quad (7)$$

Here, c denotes the latent shape embeddings extracted by the PointNet encoder. Following Karras *et al.* [8], we define $c_{\text{skip}}(t)$, $c_{\text{out}}(t)$, and c_{in} as shown below:

$$c_{\text{skip}}(t) = \frac{\sigma_{\text{data}}^2}{(t - \epsilon)^2 + \sigma_{\text{data}}^2} \quad (8)$$

$$c_{\text{out}}(t) = \frac{(t - \epsilon) \cdot \sigma_{\text{data}}}{\sqrt{t^2 + \sigma_{\text{data}}^2}} \quad (9)$$

$$c_{\text{in}}(t) = \frac{1}{\sqrt{t^2 + \sigma_{\text{data}}^2}} \quad (10)$$

Algorithm 1 Consistency Model Training

Input: $\mathcal{P}$: input point cloud
Output: $\mathcal{L}$: hybrid loss ($\mathcal{L}_{\text{Hybrid}}$)
 $\mathcal{P}' \sim \text{Uniform}(\text{normalize}(\mathcal{P}))$
 $\mathcal{P}^{(0)} = \text{Patch-Gen}(\mathcal{P}')$ $\triangleright$ simulate anomalies
 $c = \text{PointNet}(\mathcal{P}^{(0)})$
 $k = \text{training step}; \quad K = \text{total steps}$
repeat
 $N_k = \left\lceil \sqrt{(1 - \frac{k}{K})s_0^2 + \frac{k}{K}(s_1 + 1)^2} - 1 \right\rceil + 1$
 $n \sim \text{Uniform}\{0, \dots, N_k - 2\}$ $\triangleright$ sample index from schedule
 $t_n = \left[T^{1/\rho} + \frac{n}{N_k - 1}(\epsilon^{1/\rho} - T^{1/\rho}) \right]^\rho$ $\triangleright$ Karras noise level at n
 $t_{n+1} = \left[T^{1/\rho} + \frac{n+1}{N_k - 1}(\epsilon^{1/\rho} - T^{1/\rho}) \right]^\rho$
 $\epsilon \sim \mathcal{N}(0, I)$ $\triangleright$ sample gaussian noise
 $x_n = \mathcal{P}^{(0)} + t_n \cdot \epsilon$ $\triangleright$ Add noise to clean point cloud
 $x_{n+1} = x_n + \left(\frac{x_n - \mathcal{P}^{(0)}}{t_n} \right) \cdot (t_{n+1} - t_n)$
 $y = f_\theta(x_n, t_n, c)$ $\triangleright$ online network prediction
 $y^- = f_{\theta^-}(x_{n+1}, t_{n+1}, c)$ $\triangleright$ EMA network prediction
 $\mathcal{L}_{\text{Online}} = \|y - x_{\text{raw}}\|^2$
 $\mathcal{L}_{\text{Target}} = \|y^- - x_{\text{raw}}\|^2$
 $\mathcal{L}_{\text{Recons}} = \mathcal{L}_{\text{Online}} + \mathcal{L}_{\text{Target}}$
 $\mathcal{L}_{\text{CT}}(\theta, \theta^-) = \lambda(t_n) d(y, y^-)$
 $\mathcal{L}_{\text{Hybrid}} = \mathcal{L}_{\text{CT}}(\theta, \theta^-) + \lambda \cdot \mathcal{L}_{\text{Recons}}$
 $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{Hybrid}}$
 $\theta^- \leftarrow \text{stopgrad}(\mu(k) \cdot \theta^- + (1 - \mu(k)) \cdot \theta)$
 $k \leftarrow k + 1$
until convergence

where σ_{data} controls the balance between noisy and clean reconstructions. These factors ensure a smooth interpolation between noisy input and learned residual across noise levels, and enforce correct boundary behavior at $t = \epsilon$.

3.4.3. Training Objective

The underlying goal of the entire training process is to not only reconstruct consistent samples, but to also ensure that the reconstructed samples are free of anomalies. Hence, we propose a hybrid loss function as shown below:

$$\mathcal{L}_{\text{Hybrid}} = \mathcal{L}_{\text{CT}}(\theta, \theta^-) + \lambda \cdot \mathcal{L}_{\text{Recons}} \quad (11)$$

where $\mathcal{L}_{\text{CT}}(\theta, \theta^-)$ is given by:

$$\mathcal{L}_{\text{CT}}(\theta, \theta^-) = \lambda(t) d\left(f_\theta(\mathbf{x}_{t_{n+1}}, t_{n+1}, c), f_{\theta^-}(\mathbf{x}_{t_n}, t_n, c)\right) \quad (12)$$

Here, $\lambda(t)$ is a time dependent weighting function defined by:

$$\lambda(t) = \frac{1}{t^2} + \frac{1}{\sigma_{\text{data}}^2} \quad (13)$$

and d is a distance metric. For our experiments, we employ the L2 distance metric.

The reconstruction loss $\mathcal{L}_{\text{Recons}}$, is formulated as:

$$\mathcal{L}_{\text{Recons}} = \mathcal{L}_{\text{Online}} + \mathcal{L}_{\text{Target}} \quad (14)$$

$\mathcal{L}_{\text{Online}}$ and $\mathcal{L}_{\text{Target}}$ are further formulated as the Mean Squared Error (MSE) between the output of the online network f_θ and x_{raw} , and the output of the target network f_{θ^-} and x_{raw} as shown below:

$$\mathcal{L}_{\text{Online}} = \frac{1}{N} \|f_\theta(\mathbf{x}_{t_{n+1}}, t_{n+1}, c) - \mathbf{x}_{\text{raw}}\|^2 \quad (15)$$

$$\mathcal{L}_{\text{Target}} = \frac{1}{N} \|f_{\theta^-}(\mathbf{x}_{t_n}, t_n, c) - \mathbf{x}_{\text{raw}}\|^2 \quad (16)$$

In Equation 11, λ is taken to be 1 for all experiments, unless specified otherwise.

3.4.4. Adaptive Scheduling and EMA Target Network

We implement an adaptive strategy for both noise discretization and exponential moving average (EMA) decay, as shown in Algorithm 1, following the original consistency training methodology. The number of noise levels $N(k)$ at training step k is adapted from an initial value s_0 to a final value s_1 over the course of K training iterations:

$$N(k) = \left\lceil \sqrt{\frac{k}{K} ((s_1 + 1)^2 - s_0^2) + s_0^2} - 1 \right\rceil + 1 \quad (17)$$

This ensures coarse discretization in early training and finer granularity as training progresses. The target network parameters θ^- are updated using an adaptive EMA rate $\mu(k)$, defined as:

$$\mu(k) = \exp\left(\frac{s_0 \cdot \log \mu_0}{N(k)}\right) \quad (18)$$

where $\mu_0 = 0.95$ is taken to be the initial decay rate.

3.4.5. Sampling

We adopt the multi-step sampling strategy proposed in [16], enabling efficient two-step sampling. While two-step sampling yielded significantly better results than single-step sampling, increasing the number of steps beyond two yielded marginal improvements. This is further validated by our ablation studies (see section 4.6.2).

Figure 2 presents a visualization that contrasts the results obtained using two-step sampling with those achieved through single-step sampling.

4. Experimentation & Results

4.1. Dataset

We conduct experiments primarily on Anomaly-ShapeNet [10] and Real3D-AD [11].

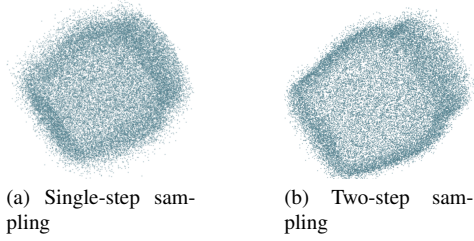


Figure 2. Comparison of reconstructions obtained via single-step sampling and two-step sampling.

Anomaly-ShapeNet contains 1,600 samples across 40 objects, each with train and test splits. Each object’s test split includes six representative anomaly types—bulge, concavity, hole, break, bending, and crack. Samples contain 8,000–30,000 points, with anomalous regions covering 1–10% of each point cloud.

Real3D-AD is a widely used real-world benchmark for industrial point cloud anomaly detection. It contains 1,254 samples from 12 object categories, each with four clean prototypes, and over 100 test samples with annotated anomalous and normal regions, making it an important benchmark for validating performance on industry-grade, high-resolution samples.

Hyperparameter	Value
Initial learning rate	2×10^{-4}
Final learning rate	5×10^{-6}
Number of steps	800K
s_0	2
s_1	1025
μ_0	0.95
Training iterations	800K
σ_{data}	0.5
ϵ	0.002
T	80.0
ρ	7
Batch size	128

Table 1. Hyperparameter values used in training.

4.2. Experimental Setup & Implementation Details

4.2.1. Hyperparameter Configuration

The final set of hyperparameters were determined through an iterative refinement process, beginning with a baseline configuration adapted from prior work and subsequently optimized through experimentation.

In alignment with the previous work on consistency models [16], and based on our experimentation, the learning rate was chosen to start from $2e-4$ and was kept constant for the first 10,000 training iterations. It was then annealed

to $5e-6$ across 790,000 subsequent iterations and was kept constant at $5e-6$ for the final 10,000 steps. Thus, each object was trained for a total of 800,000 iterations.

Prior to training, we normalized each point cloud by translating its center of gravity to the origin and scaling it to lie within the range of -1 to 1. We then computed the standard deviation of the normalized dataset and set σ_{data} to 0.5 accordingly. A comprehensive summary of the final hyperparameter configuration is provided in Table 1.

4.2.2. Training Details

Our model was trained on 2x NVIDIA A100 (40GB) GPUs, requiring approximately 3.5 GPU hours per category on the Anomaly ShapeNet dataset and approximately 5.5 hrs per category on the Real3D-AD dataset. We employ mixed-precision training, alternating between BF16 and FP32, to further improve training efficiency.

4.2.3. On-Device Evaluation

We benchmark R3D-AD and CM3D-AD (ours) on the Raspberry Pi 4 and the Jetson Nano, and report FLOPs, runtime latency, RAM usage, CPU usage, and GPU usage.

Figure 3 presents a Pareto analysis of four efficiency metrics versus AUROC for CM3D-AD and R3D-AD on both Raspberry Pi 4 and Jetson Nano (2 GB). Across both devices, CM3D-AD consistently occupies a more favorable efficiency–accuracy operating region, enabled by non-iterative single-step inference through consistency models. On Raspberry Pi 4 (CPU-only), CM3D-AD achieves an approximately $80\times$ latency speedup over R3D-AD and reduces RAM usage by about 180 MB. On Jetson Nano, CM3D-AD maintains the same trend, with an approximately $53\times$ speedup and about 467 MB lower RAM usage. Importantly, these efficiency gains are obtained without a meaningful degradation in AUROC.

Table 2 provides the corresponding tabulated values for these on-device measurements and additionally reports hardware-independent model characteristics, namely model size and FLOPs. As expected, R3D-AD benefits from Jetson Nano’s CUDA-enabled acceleration relative to its Raspberry Pi 4 runtime; however, CM3D-AD remains substantially more efficient on both platforms.

4.3. Evaluation Metrics

In line with prior approaches for fair comparison, we evaluate our model on both Anomaly-ShapeNet and Real3D-AD and compute image-level AUROC (I-AUROC) for each object category. First, a nearest neighbor scorer is employed which performs an FAISS-NN search [6] to search for the nearest neighbor of the test point clouds in the training set and accordingly assigns anomaly scores. Based on this scoring, the point cloud is assigned a label of being anomalous, or anomaly free.

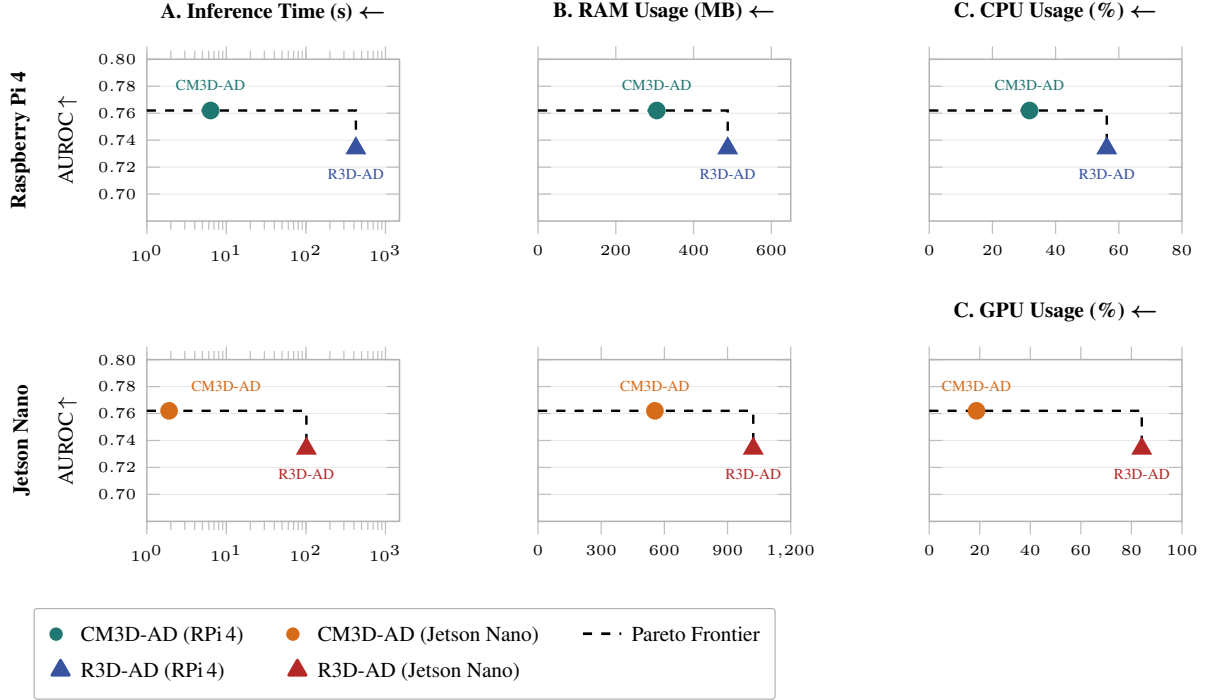


Figure 3. Pareto analysis of AUROC vs. on-device runtime cost on Raspberry Pi 4 (top, ●/▲) and Jetson Nano (bottom, ●/▲). The dashed staircase marks each platform’s Pareto frontier. CM3D-AD achieves higher AUROC at lower cost across all axes on both platforms. AUROC values reflect Anomaly-ShapeNet performance; efficiency metrics are hardware-measured and dataset-independent.

Model	Model Size (↓)	FLOPs ($\times 10^9$) (↓)	Hardware	Time (↓)	Mem. (RAM) (↓)	CPU Usage (↓)	GPU Usage (↓)
CM3D-AD	20.91 MB	1.721	Raspberry Pi 4	6.341s	305.52 MB	31.75%	—
			Jetson Nano	1.905s	555 MB	23.70%	18.7%
R3D-AD [22]	17.01 MB	135.039	Raspberry Pi 4	424.510s	487.93 MB	56.20%	—
			Jetson Nano	101.320s	1022 MB	25.10%	84.10%

Table 2. Comparison of model complexity and on-device inference performance of CM3D-AD (ours) against state-of-the-art methods on Raspberry Pi 4 and Jetson Nano. The best results are highlighted in **bold**.

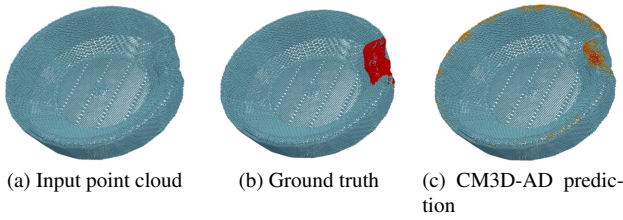


Figure 4. Qualitative visualization of anomaly localization for an Anomaly-ShapeNet sample: (a) input point cloud, (b) ground-truth anomaly localization, and (c) anomaly map predicted by CM3D-AD (Ours).

The AUROC score of these predictions against the corresponding ground truth labels is then calculated to obtain the final image level AUROC metric. An AUROC score of 0.5

indicates that the prediction made by the model is a random guess, and the closer the score is to 1, the more confident is the model’s prediction.

4.4. Results

We primarily compare against R3D-AD [22], the state-of-the-art diffusion-based method, to evaluate consistency models as an efficient alternative to diffusion-based reconstruction; additional baselines are included for broader context. Results in Table 3 and Table 4 show competitive performance, including a 1.3% AUROC improvement over R3D-AD on Anomaly-ShapeNet. On Real3D-AD, we observe a slight AUROC degradation ($\approx 0.6\%$). However, by achieving these scores in just two sampling steps, with massive compute and latency gains, our model enables rapid anomaly detection in resource-constrained environ-

Method	ashtray0	bag0	bottle0	bottle1	bottle3	bow10	bow11	bow12	bow13	bow14	bow15	bucket0	bucket1	cap0
BTF(Raw)	0.578	0.410	0.597	0.510	0.568	0.564	0.264	0.525	0.385	0.664	0.417	0.617	0.321	0.668
BTF(FPFH)	0.420	0.546	0.344	0.546	0.322	0.509	0.668	0.510	0.490	0.609	0.699	0.401	0.633	0.618
M3DM	0.577	0.537	0.574	0.637	0.541	0.634	0.663	0.684	0.617	0.464	0.409	0.309	0.501	0.557
Patchcore(FPFH)	0.587	0.571	0.604	0.667	0.572	0.504	0.639	0.615	0.537	0.494	0.558	0.469	0.551	0.580
Patchcore(PointMAE)	0.591	0.601	0.513	0.601	0.650	0.523	0.629	0.458	0.579	0.501	0.593	0.593	0.561	0.589
CPMF	0.353	0.643	0.520	0.482	0.405	0.783	0.639	0.625	0.658	0.683	0.685	0.482	0.601	0.601
Reg3D-AD	0.597	0.706	0.486	0.695	0.525	0.671	0.525	0.490	0.348	0.663	0.593	0.610	0.752	0.693
IMRNet	0.671	0.660	0.552	0.700	0.640	0.681	0.702	0.685	0.599	0.676	<u>0.710</u>	0.580	0.771	0.737
R3D-AD	<u>0.833</u>	<u>0.720</u>	0.733	<u>0.737</u>	0.781	<u>0.819</u>	0.778	<u>0.741</u>	<u>0.767</u>	<u>0.744</u>	0.656	<u>0.683</u>	<u>0.756</u>	0.822
CM3D-AD (Ours)	0.838	0.734	<u>0.731</u>	0.743	<u>0.717</u>	0.821	<u>0.730</u>	0.780	0.786	0.820	0.740	0.783	0.723	<u>0.766</u>

Method	cap3	cap4	cap5	cup0	cup1	eraser0	headset0	headset1	helmet0	helmet1	helmet2	helmet3	jar0
BTF(Raw)	0.527	0.468	0.373	0.403	0.521	0.525	0.378	0.515	0.553	0.349	0.602	0.526	0.420
BTF(FPFH)	0.522	0.520	0.586	0.586	0.610	0.719	0.520	0.490	0.571	0.719	0.542	0.444	0.424
M3DM	0.423	<u>0.777</u>	0.639	0.539	0.556	0.627	0.577	0.617	0.526	0.427	0.623	0.374	0.441
Patchcore(FPFH)	0.453	0.757	0.790	0.600	0.586	0.657	0.583	0.637	0.546	0.484	0.425	0.404	0.472
Patchcore(PointMAE)	0.476	0.727	0.538	0.610	0.556	0.677	0.591	0.627	0.556	0.552	0.447	0.424	0.483
CPMF	0.551	0.553	0.697	0.497	0.499	0.689	0.643	0.458	0.555	0.589	0.462	0.520	0.610
Reg3D-AD	0.725	0.643	0.467	0.510	0.538	0.343	0.537	0.610	0.600	0.381	0.614	0.367	0.592
IMRNet	<u>0.775</u>	0.652	0.652	0.643	0.757	0.548	0.720	0.676	0.597	0.600	<u>0.641</u>	0.573	0.780
R3D-AD	0.730	0.681	0.670	0.776	0.757	0.890	<u>0.738</u>	0.795	<u>0.757</u>	<u>0.720</u>	0.633	<u>0.707</u>	0.838
CM3D-AD (Ours)	0.765	0.714	<u>0.760</u>	<u>0.743</u>	<u>0.730</u>	<u>0.831</u>	0.771	<u>0.761</u>	0.787	0.731	0.681	0.723	<u>0.788</u>

Method	microphone0	shelf0	tap0	tap1	vase0	vase1	vase2	vase3	vase4	vase5	vase7	vase8	vase9	Average
BTF(Raw)	0.563	0.164	0.525	0.573	0.531	0.549	0.410	0.717	0.425	0.585	0.448	0.424	0.564	0.493
BTF(FPFH)	0.671	0.609	0.560	0.546	0.342	0.219	0.546	0.699	0.510	0.409	0.518	0.668	0.268	0.528
M3DM	0.357	0.564	0.754	0.739	0.423	0.427	0.737	0.439	0.476	0.317	0.657	0.663	0.663	0.552
Patchcore(FPFH)	0.388	0.494	<u>0.753</u>	0.766	0.455	0.423	0.721	0.449	0.506	0.417	0.693	0.662	0.660	0.568
Patchcore(PointMAE)	0.488	0.523	0.458	0.538	0.447	0.552	0.741	0.460	0.516	0.579	0.650	0.663	0.629	0.562
CPMF	0.509	0.685	0.359	0.697	0.451	0.345	0.582	0.582	0.514	0.618	0.397	0.529	0.609	0.559
Reg3D-AD	0.414	0.688	0.676	0.641	0.533	0.702	0.605	0.650	0.500	0.520	0.462	0.620	0.594	0.572
IMRNet	0.755	0.603	0.676	0.696	0.533	0.757	0.614	0.700	0.524	0.676	0.635	0.630	0.594	0.659
R3D-AD	<u>0.762</u>	<u>0.696</u>	0.736	<u>0.900</u>	0.788	0.729	<u>0.752</u>	<u>0.742</u>	<u>0.630</u>	<u>0.757</u>	<u>0.771</u>	<u>0.721</u>	<u>0.718</u>	0.749
CM3D-AD (Ours)	0.773	0.710	0.738	0.903	<u>0.784</u>	<u>0.750</u>	0.770	0.760	0.698	0.770	0.794	0.753	0.765	0.762

Table 3. Image-level anomaly detection AUROC across object categories on the Anomaly-ShapeNet dataset. Best results are in **bold** and second best are underlined.

Method	Airplane	Candybar	Car	Chicken	Diamond	Duck	Fish	Gemstone	Seahorse	Shell	Starfish	Toffees	Average
BTF(Raw)	0.730	0.539	0.647	0.789	0.707	0.691	0.602	0.686	0.596	0.396	0.530	0.703	0.635
BTF(FPFH)	0.520	0.630	0.560	0.432	0.545	0.784	0.549	0.648	0.779	<u>0.754</u>	0.575	0.462	0.603
M3DM	0.434	0.552	0.541	0.683	0.602	0.433	0.540	0.644	0.495	0.694	0.551	0.450	0.552
Patchcore(FPFH)	0.882	0.541	0.590	<u>0.837</u>	0.574	0.546	0.675	0.370	0.505	0.589	0.441	0.565	0.593
Patchcore(PointMAE)	0.726	0.663	0.498	0.827	0.783	0.489	0.630	0.374	0.539	0.501	0.519	0.585	0.595
CPMF	0.701	0.552	0.551	0.504	0.523	0.582	0.558	0.589	<u>0.729</u>	0.653	0.700	0.390	0.586
Reg3D-AD	0.716	0.685	0.697	0.852	<u>0.900</u>	0.584	0.915	0.417	0.762	0.583	0.506	0.827	0.704
IMRNet	0.762	0.755	0.711	0.780	0.905	0.517	<u>0.880</u>	0.674	0.604	0.665	0.674	<u>0.774</u>	0.725
R3D-AD	0.772	0.696	<u>0.713</u>	0.714	0.685	0.909	0.692	0.665	0.720	0.840	0.701	0.703	0.734
CM3D-AD (Ours)	<u>0.781</u>	<u>0.716</u>	0.718	0.723	0.712	<u>0.804</u>	0.720	<u>0.681</u>	0.724	0.734	0.713	0.706	<u>0.728</u>

Table 4. Image-level anomaly detection AUROC across object categories on the Real3D-AD dataset. Best results are in **bold** and second best are underlined.

ments and helps bridge the gap between academic research and industrial application.

In addition, Figure 4 provides a qualitative visualization of anomaly localization on the *ashtray0* category from Anomaly-ShapeNet, contrasting the ground-truth localiza-

tion with the anomaly map predicted by CM3D-AD.

4.5. Out-of-distribution Performance

To assess the generalizability of the proposed model, we conduct systematic cross-category and cross-dataset eval-

Train Dataset	Train Category	Test Dataset	Test Category	I-AUROC	CD
Anomaly-Shapenet	ashtray0	Anomaly-Shapenet	bowll	0.814815	-
Anomaly-Shapenet	headset0	Anomaly-Shapenet	headset1	0.724825	-
Real3DAD	airplane	ShapeNetCore.v2	airplane	-	0.0164027
Real3DAD	car	ShapeNetCore.v2	car	-	0.0324160

Table 5. Generalization capability on unseen data.

Loss Function	Steps	Learning Rate	I-AUROC
$\mathcal{L}_{CT} + \mathcal{L}_{Online}$	800K	$2 \times 10^{-4} \rightarrow 5 \times 10^{-6}$	0.725
$\mathcal{L}_{CT} + \mathcal{L}_{Target}$	800K	$2 \times 10^{-4} \rightarrow 5 \times 10^{-6}$	0.689
$\mathcal{L}_{Hybrid}$ (11)	800K	$2 \times 10^{-4} \rightarrow 5 \times 10^{-6}$	0.762

Table 6. Ablation study comparing different loss configurations for our method. I-AUROC scores are averaged across all object categories on **Anomaly-ShapeNet**. The highest value is highlighted in **bold**.

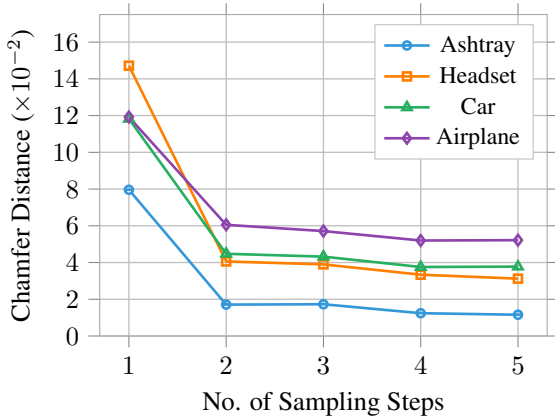


Figure 5. Comparison of **Chamfer Distance (CD)** across sampling steps.

uations, as summarized in Table 5. The model maintains strong and consistent performance across categories within the same dataset as well as across different datasets, achieving I-AUROC scores of up to 0.815 in certain settings. To further quantify generalization on known categories, we evaluate our model trained on Real3D-AD on a similar category from ShapeNetCore.v2 dataset [4]. Since ShapeNetCore.v2 does not contain anomalous samples or anomaly labels, AUROC is not applicable; therefore, we report Chamfer Distance (CD) for this experiment.

4.6. Ablation Studies

4.6.1. Analysis of Hybrid Loss Formulation

The hybrid loss in Eq. (11) comprises three terms: the consistency loss $\mathcal{L}_{CT}(\theta, \theta^-)$ and two reconstruction losses, $\mathcal{L}_{Online}$ and $\mathcal{L}_{Target}$, which supervise the online and EMA

target networks against the anomaly-free point cloud x_{raw} . While the role of $\mathcal{L}_{CT}$ is to enforce agreement between the two networks, the necessity of retaining *both* reconstruction terms is less obvious.

To study this, we train two reduced variants using only $\{\mathcal{L}_{Online}, \mathcal{L}_{CT}\}$ or $\{\mathcal{L}_{Target}, \mathcal{L}_{CT}\}$, while keeping all other hyperparameters fixed. As shown in Table 6, both reduced variants underperform the full three-term objective in terms of I-AUROC, indicating that directly supervising both networks is more effective than relying on consistency alone.

4.6.2. Effect of Multi-Step Sampling

As shown in subsection 3.4.5, two-step sampling outperforms single-step sampling. We therefore investigate whether increasing the number of sampling steps beyond two yields further gains in reconstruction quality. Although Song *et al.* [16] report only marginal improvements beyond two steps, we validate this behavior on 3D point cloud data.

Specifically, we evaluate four checkpoints (two per dataset) on test samples from both Anomaly-ShapeNet [10] and Real-3D-AD [11], and compare reconstruction error under single-step, two-step, and multi-step sampling, as shown in Figure 5. Our findings are consistent with [16], confirming that gains beyond two sampling steps are marginal.

5. Conclusion

This paper addresses a central barrier to the industrial adoption of recent 3D point cloud anomaly detection methods—the latency bottleneck—by reformulating the task as a single-step manifold projection problem. We show that conditionally guided consistency models offer an effective, practical, and increasingly necessary alternative to diffusion-based pipelines, delivering comparable or better reconstruction quality with substantially lower memory usage and up to an $80\times$ speedup. We further evaluate the proposed model against R3D-AD, the state-of-the-art diffusion baseline, on two representative edge platforms: Raspberry Pi 4 (8 GB) and Jetson Nano (2 GB). Across both devices, results show that consistency models make practical edge deployment feasible, with CM3D-AD meeting real-world efficiency and latency requirements while maintaining competitive anomaly detection performance.

References

- [1] J. Bae, J. H. Lee, and S. Kim. Pni: Industrial anomaly detection using position and neighborhood information. In *ICCV*, 2023. 2
- [2] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger. Improving unsupervised defect segmentation by applying structural similarity to autoencoders. In *VISIGRAPP*, 2019. 2
- [3] Yunkang Cao, Xiaohao Xu, and Weiming Shen. Complementary pseudo multimodal feature for point cloud anomaly detection. *Pattern Recognition*, 156:110761, 2024. 2
- [4] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015. 8
- [5] Gene Chou, Yuval Bahat, and Felix Heide. Diffusion-sdf: Conditional generative modeling of signed distance functions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2262–2272, 2023. 1
- [6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *arXiv preprint*, 2024. 5
- [7] D. Gudovskiy, S. Ishizaka, and K. Kozuka. Cflow-ad: Real-time unsupervised anomaly detection with localization via conditional normalizing flows. In *WACV*, 2022. 2
- [8] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Proceedings of NeurIPS*, 2022. 3
- [9] D. Kim, C. Park, S. Cho, and S. Lee. Fapm: Fast adaptive patch memory for real-time industrial anomaly detection. In *ICASSP*, 2023. 2
- [10] Wenqiao Li, Xiaohao Xu, Yao Gu, Bozhong Zheng, Shenghua Gao, and Yingna Wu. Towards scalable 3d anomaly detection and localization: A benchmark via 3d anomaly synthesis and a self-supervised learning network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22207–22216, 2024. 1, 2, 4, 8
- [11] Jiawei Liu, Guangyao Xie, Xinyu Li, Jianqiang Wang, Yuxin Liu, Chao Wang, Feng Zheng, et al. Real3d-ad: A dataset of point cloud anomaly detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 1, 4, 8
- [12] S. Luo and W. Hu. Diffusion probabilistic models for 3d point cloud generation. In *CVPR*, 2021. 1, 2
- [13] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. *arXiv preprint*, 2016. 2, 3
- [14] Karl Roth, Lavanya Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2
- [15] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021. 2
- [16] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency models. In *ICML*, 2023. 1, 2, 3, 4, 5, 8
- [17] M. Tailanian, Á. Pardo, and P. Musé. U-flow: A u-shaped normalizing flow for anomaly detection with unsupervised threshold. *arXiv preprint*, 2022. 2
- [18] Yiming Wang, Jiawei Peng, Jiawei Zhang, Ran Yi, Yanan Wang, and Chao Wang. Multimodal industrial anomaly detection via hybrid fusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
- [19] J. Yu, Y. Zheng, X. Wang, W. Li, Y. Wu, R. Zhao, and L. Wu. Fastflow: Unsupervised anomaly detection and localization via 2d normalizing flows. *arXiv preprint*, 2021. 2
- [20] V. Zavrtanik, M. Kristan, and D. Skočaj. Draem: A discriminatively trained reconstruction embedding for surface anomaly detection. In *ICCV*, 2021. 2
- [21] V. Zavrtanik, M. Kristan, and D. Skočaj. Reconstruction by inpainting for visual anomaly detection. *Pattern Recognition*, 2021. 2
- [22] Z. Zhou, L. Wang, N. Fang, Z. Wang, L. Qiu, and S. Zhang. R3d-ad: Reconstruction via diffusion for 3d anomaly improving. In *ECCV*, 2024. 1, 2, 6